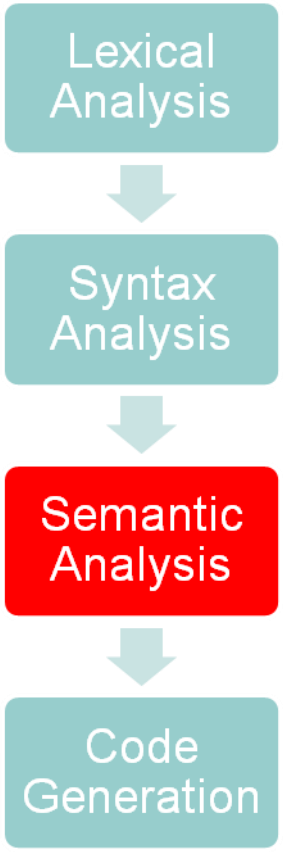
Unit 3.1: Symbol table & scope management

Nội dung

- Tổng quan về phân tích ngữ nghĩa
- Xây dựng bảng ký hiệu
- Quản lý phạm vi

Semantic analysis



- Syntax analysis only guarantees the syntactical correctness of given programs, i.e. conform with the grammar rules of the PL.
- Correctness of a program should have more checks:
  - Has a variable “x” been declared?
  - Where a variable “x” has been declared?
  - Is “x” a variable or a function/procedure?
  - Is “a+b” type-correct?
  - …

Tasks of semantic analysis

- Management of identifiers
  - Constant identifiers
  - Variable identifiers
  - Type identifiers
  - Function/procedure identifiers
- Check semantics rules
  - Scope rules
  - Type consistency

Symbol table (1)

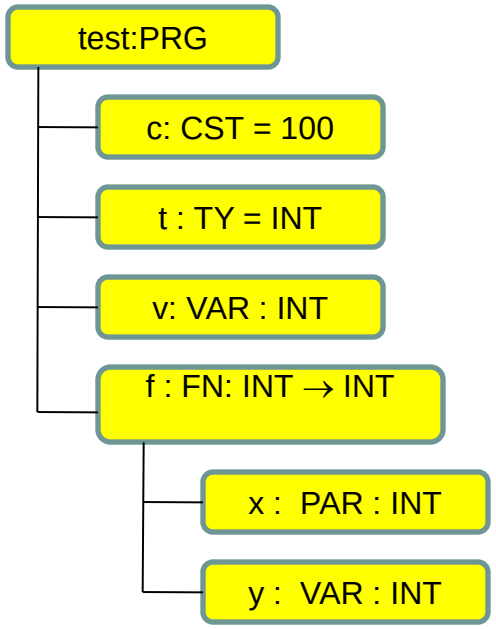
- Store semantic properties of identifiers (symbols)
  - Constant: {name, type, value}
  - User-define type: {name, actual type}
  - Variable: {name, type}
  - Function: {name, formal parameters, return type, local definitions}
  - Procedure: {name, formal parameters, local definitions)
  - Formal parameter: {name, type, pass-by-value/pass-by-reference}

Symbol table (2)

- Symbols can be represented in a hierarchical structure

```
PROGRAM test;
CONST c = 100;
TYPE t = Integer;
VAR v : t;
FUNCTION f(x : t) : t;
  VAR y : t;
BEGIN
  y := x + 1;
  f := y;
END;

BEGIN
  v := 1;
  WriteI(f(v));
END.
```



Symbol table implementation

- Initialization and clean-up
- Adding symbols at their declaration:
  - Constant declaration
  - Type declaration
  - Variable declaration
  - Function/procedure declaration
  - Parameter declaration

Symbol table implementation (1)

```
// Bảng ký hiệu
struct SymTab_ {
  // Chương trình chính
  Object* program;
  // Phạm vi hiện tại
  Scope* currentScope;
  // Các đối tượng toàn cục như
  // hàm WRITEI, WRITEC, WRITELN
  // READI, READC
  ObjectNode *globalObjectList;
};
```

```
// Phạm vi của một block
struct Scope_ {
  // Danh sách các đối tượng trong
  // block
  ObjectNode *objList;
  // Hàm, thủ tục, chương trình
  // tương ứng block
  Object *owner;
  // Phạm vi bao ngoài
  struct Scope_ *outer;
};
```

Symbol table implementation (2)

- Current block which is being processed: `currentScope`
- Whenever starting parsing a function/procedure, we need update the `currentScope` by using:

```
void enterBlock(Scope* scope);
```

- Whenever finishing parsing a function/procedure, we need change `currentScope` to the one of outer scope by using

```
void exitBlock(void);
```

- Register an object (constant, type, variable, function, procedure, parameter) to the current block:

```
void declareObject(Object* obj);
```

Object

```
// Object kinds

enum ObjectKind {
  OBJ_CONSTANT,
  OBJ_VARIABLE,
  OBJ_TYPE,
  OBJ_FUNCTION,
  OBJ_PROCEDURE,
  OBJ_PARAMETER,
  OBJ_PROGRAM
};
```

```
// Object

struct Object_ {
  char name[MAX_IDENT_LEN];
  enum ObjectKind kind;
  union {
    ConstantAttributes* constAttrs;
    VariableAttributes* varAttrs;
    TypeAttributes* typeAttrs;
    FunctionAttributes* funcAttrs;
    ProcedureAttributes* procAttrs;
    ProgramAttributes* progAttrs;
    ParameterAttributes* paramAttrs;
  };
};
```

Various kinds of objects (1)

```
struct ConstantAttributes_ {
  ConstantValue* value;
};

struct VariableAttributes_ {
  Type *type;
  // Phạm vi của biến (sử dụng cho pha sinh mã)
  struct Scope_ *scope;
};

struct TypeAttributes_ {
  Type *actualType;
};

struct ParameterAttributes_ {
  // Tham biến hoặc tham trị
  enum ParamKind kind;
  Type* type;
  struct Object_ *function;
};
```

Semantic attributes of constant and type

```
// Phân loại kiểu

enum TypeClass {
  TP_INT,
  TP_CHAR,
  TP_ARRAY
};

struct Type_ {
  enum TypeClass typeClass;
  // Chỉ sử dụng cho kiểu mảng
  int arraySize;
  struct Type_ *elementType;
};
```

```
// Hằng

struct ConstantValue_ {
  enum TypeClass type;
  union {
    int intValue;
    char charValue;
  };
};
```

Creating constant and type objects

- Các hàm tạo kiểu

```
Type* makeIntType(void);
Type* makeCharType(void);
Type* makeArrayType(int arraySize, Type* elementType);
Type* duplicateType(Type* type)
```

- Các hàm tạo giá trị hằng số

```
ConstantValue* makeIntConstant(int i);
ConstantValue* makeCharConstant(char ch);
ConstantValue*
  duplicateConstantValue(ConstantValue* v);
```

Function, procedure, and program (2)

```
struct ProcedureAttributes_ {
  struct ObjectNode_ *paramList;
  struct Scope_* scope;
};

struct FunctionAttributes_ {
  struct ObjectNode_ *paramList;
  Type* returnType;
  struct Scope_ *scope;
};

struct ProgramAttributes_ {
  struct Scope_ *scope;
};

// Note: A parameter object is stored in both paramList and objList of
   the scope
```

Implementation: compile

```
int compile(char *fileName) {
  …
  // Khởi tạo bảng ký hiệu
  initSymTab();
  // Dịch chương trình, điền thông tin vào bảng ký hiệu
  compileProgram();
  // In chương trình để kiểm tra kết quả
  printObject(symtab->program,0);
  // Giải phóng bảng ký hiệu
  cleanSymTab();
  …
}
```

Symbol table initialization

- Program object will be registered at:

```
void compileProgram(void);
```

- Setting up current block: `enterBlock()`
- Call `exitBlock()` after successfully compiled the program

Adding constants

- Constant objects are created at `compileBlock()`
- Constant values are obtained from return values of:

```
ConstantValue* compileConstant(void)
ConstantValue* compileConstant2(void)
```

- If constant value is a constant identifier, we need to lookup to get the actual value
- Register constant object to the symbol table: `declareObject`

compileConstant2

```
ConstantValue* compileConstant2(void) {
  ConstantValue* constValue;
  Object* obj;

  switch (lookAhead->tokenType) {
  case TK_NUMBER:
    eat(TK_NUMBER);
    constValue = makeIntConstant(currentToken->value);
    break;
  case TK_IDENT:
    eat(TK_IDENT);

    obj = lookupObject(currentToken->string);
    if ((obj != NULL) && (obj->kind == OBJ_CONSTANT) && (obj->constAttrs->value->type == TP_INT))
      constValue = duplicateConstantValue(obj->constAttrs->value);
    else
      error(ERR_UNDECLARED_INT_CONSTANT,currentToken->lineNo, currentToken->colNo);
    break;
  default:
    error(ERR_INVALID_CONSTANT, lookAhead->lineNo, lookAhead->colNo);
    break;
  }
  return constValue;
}
```

Adding types

- Type objects are created at `compileBlock2()`
- Actual type can be obtained by

  `Type* compileType(void)`

  - Type identifier should be replaced by its actual type
- Register a type object to the symbol table: `declareObject`

Adding variables

- Variable objects are created at `compileBlock3()`
- Type of the variable object can be obtained by:

  `Type* compileType(void)`
- Current scope is assigned to the scope of the variable object for code generation
- Register variable object to the symbol table: `declareObject`

Adding functions

- Function objects are created at `compileFuncDecl()`
- Getting semantic attributes of a function object:
  - Parameters: `compileParams`
  - Return type: `compileBasicType`
  - Scope: current scope
- Note: changing current scope to the scope of the function before compiling its local definition

Adding procedures

- Procedure objects are created at `compileProcDecl()`
- Getting semantic attributes of a function object:
  - Parameters: `compileParams`
  - Scope: current scope
- Note: changing current scope to the scope of the procedure before compiling its local definition

Adding formal parameter

- Parameter objects are created at `compileParam()`
- Semantic properties:
  - Type
  - Pass-by-reference (PARAM_REFERENCE) pass-by-value (PARAM_VALUE)
- Note: parameter objects are added to both paramList and to the scope of the function/procedure

Scope management

- Checking the duplicate declaration
- Checking the references to the symbols

Checking legal name (1)

- A legal name is a legal identifier which has not been declared in the current scope.

```
void checkFreshIdent(char *name);
```

Checking legal name (2)

- Name checking is proceeded at:
  - Constant declaration
  - Type declaration
  - Variable declaration
  - Parameter declaration
  - Function declaration
  - Procedure declaration

Checking declared constants

- Declared constant checking is proceeded when we have a reference to this constant:
  - compileConstant
  - CompileConstant2
- Note: if constant name is not found in the current scope, we need to look for it in outer scopes
- Constant value is non-shared: we need to duplicate this value for each constant object → duplicateConstantValue

Checking declared type

- Declared type checking is proceeded when we refer to a type: compileType
- If type identifier can not be found in the current scope, we need to look for it in outer scopes
- Actual types are non-shared. We need to duplicate when making a type object
  - duplicateType

Checking declared variables

- Declared variable checking is proceeded at:
  - Assignments
  - For-statements
  - Factors
- If a variable name is not found in the current scope, wee need to look up for it in the outer scope

Checking declared variables

- When an identifier appeared on the LHS of an assignment or as a factor, this would be:
  - Name of the current function, or
  - A declared parameter
  - A declared variable
    - If this variable has array type, there should be indices followed

Checking declared function

- Declared function checking is proceeded at
  - LHS of an assignment (current function)
  - factor (followed by parameters)
- Some global functions: READC, READI

Function name on LHS of assignments

```
function f(x:integer):integer
 .....
        function g(y: integer):integer
        begin
        f:= y*y  (* lenh gan cho ten ham khac- ko hop le*)
        end;
.....
```

Checking declared procedure

- Declared procedure checking is proceeded at
  - Call-statement
- If the procedure name is not found in the current scope, we must look it up in the outer scopes
- Global procedures: WRITEI, WRITEC, WRITELN

Semantic error codes

- ERR_UNDECLARED_IDENT
- ERR_UNDECLARED_CONSTANT
- ERR_UNDECLARED_TYPE
- ERR_UNDECLARED_VARIABLE
- ERR_INVALID_RETURN
- ERR_UNDECLARED_PROCEDURE
- ERR_DUPLICATE_IDENT

lookupObject

```
Object* lookupObject(char *name)

{  Scope* scope = symtab->currentScope;
  Object* obj;
  while (scope != NULL) {
    obj = findObject(scope->objList, name);
    if (obj != NULL) return obj;
    scope = scope->outer;
  }
  obj = findObject(symtab->globalObjectList, name);
  if (obj != NULL) return obj;
  return NULL;
}
```

Tasks for you!!!

- Filling up TODO marks in symtab.c
- Understanding changes in the parser
- Filling up TODO marks for registering objects
- Experimenting with examples

Nhiệm vụ thực hành

- Lập trình cho các hàm sau trong tệp semantics.c
  - checkDeclaredIdent
  - checkDeclaredVariable
  - checkDeclaredProcedure
  - checkDeclaredLValueIdent
- Bổ sung các đoạn code vào những chỗ có đánh dấu TODO hoặc những chỗ cần thiết để thực hiện các công việc kiểm tra khi đối tượng được khai báo hay sử dụng
- Biên dịch và thử nghiệm với các ví dụ mẫu